

Harkirat Singh

Abstract

Hospital readmissions are a major driver of healthcare cost and patient risk, making early identification of high-risk patients a critical challenge. This project applies an end-to-end machine learning pipeline to predict hospital readmission for diabetic patients using a large, real-world clinical dataset containing over 100,000 encounters. The workflow includes exploratory data analysis, data cleaning, feature engineering, handling class imbalance, and high-dimensional categorical encoding. Multiple models—including Naive Bayes, Decision Trees, Random Forests, and ensemble techniques—were trained and evaluated using accuracy, precision, recall, and F1-score. Dimensionality reduction with PCA and preprocessing pipelines were also implemented to assess scalability and model robustness. Results demonstrate the trade-offs between model interpretability, recall for high-risk patients, and overall predictive performance, with Random Forest achieving the most balanced results. This project highlights practical applied machine learning skills, model evaluation reasoning, and real-world considerations relevant to healthcare analytics and production-ready ML systems.

Machine Learning–Based Prediction of Hospital Readmissions

```
In [74]: import numpy as np # numerical operations
import pandas as pd # data handling library
import matplotlib.pyplot as plt
import seaborn as sns
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split # split dataset
from sklearn.naive_bayes import GaussianNB # import model
from sklearn.metrics import accuracy_score # accuracy metric
from sklearn.tree import DecisionTreeClassifier # import model
from sklearn.ensemble import RandomForestClassifier # import model
from sklearn.compose import ColumnTransformer # column-wise preprocessing
from sklearn.pipeline import Pipeline # build pipeline
from sklearn.preprocessing import OneHotEncoder # encode categoricals
from sklearn.preprocessing import StandardScaler # scale features
from sklearn.impute import SimpleImputer # impute missing values
from sklearn.decomposition import TruncatedSVD # SVD for sparse PCA
from sklearn.neighbors import KNeighborsClassifier # import KNN
from sklearn.metrics import accuracy_score # accuracy
from sklearn.ensemble import BaggingClassifier # import bagging
from sklearn.tree import DecisionTreeClassifier # decision tree
from sklearn.ensemble import AdaBoostClassifier # import boosting
from sklearn.ensemble import StackingClassifier # stacking
from sklearn.linear_model import LogisticRegression # meta learner
from sklearn.naive_bayes import GaussianNB # NB
from sklearn.metrics import precision_score, recall_score, f1_score

# load main dataset
df = pd.read_csv("diabetic_data.csv") # read diabetic data

# load ID mapping file (optional for later parts)
ids_map = pd.read_csv("IDS_mapping.csv") # read ID mapping info

In [75]: print("Dataset shape:", df.shape) # rows, columns count

Dataset shape: (101766, 50)
```

The dataset was successfully loaded into a pandas data frame from the provided CSV file. This dataset contains hospital encounter information for diabetic patients. Also, the dataset contains 101,766 rows, patient encounters, and 50 columns, features and identifiers.

```
In [76]: df.head() # display first five rows
```

Out[76]:

	encounter_id	patient_nbr	race	gender	age	weight	admission_type_id	discharge_disposition_id	admission_source_id	time_in_hospital	...	c
0	2278392	8222157	Caucasian	Female	[0-10)	?	6	25	1	1	...	
1	149190	55629189	Caucasian	Female	[10-20)	?	1	1	7	3	...	
2	64410	86047875	AfricanAmerican	Female	[20-30)	?	1	1	7	2	...	
3	500364	82442376	Caucasian	Male	[30-40)	?	1	1	7	2	...	
4	16680	42519267	Caucasian	Male	[40-50)	?	1	1	7	1	...	

5 rows × 50 columns

The first five rows of the dataset show sample patient records, confirming that the dataset loaded correctly and contains patient demographics, encounter information, diagnoses, and medication fields.

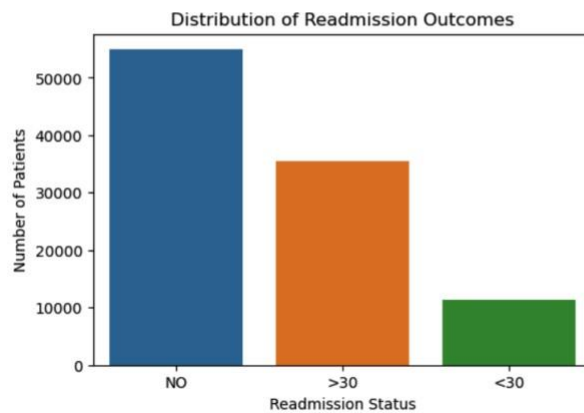
Data Understanding & Exploration

Background Understanding

The dataset contains hospital encounter records for diabetic patients. The goal is to analyze patient characteristics and identify patterns related to hospital readmission.

Readmission Outcome Distribution

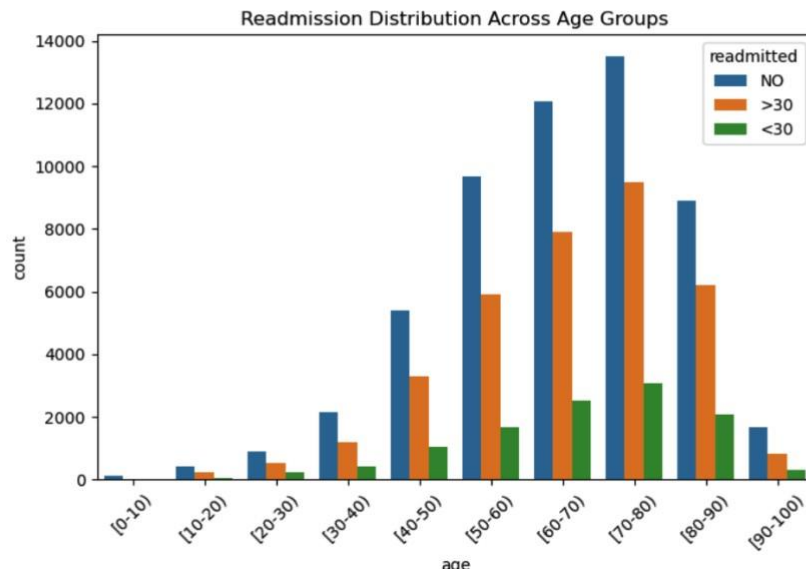
```
In [77]: plt.figure(figsize=(6,4)) #set thr plot demensions
sns.countplot(x=df['readmitted']) #create a frequency bar chart
plt.title("Distribution of Readmission Outcomes") #add a main heading
plt.xlabel("Readmission Status") #Label the horizontal axis
plt.ylabel("Number of Patients") #Label the verticle axis
plt.show() # display the visuals
```



Most patients are not readmitted, while fewer are readmitted after 30 days and the smallest group within 30 days, showing class imbalance.

Readmission Across Age Groups

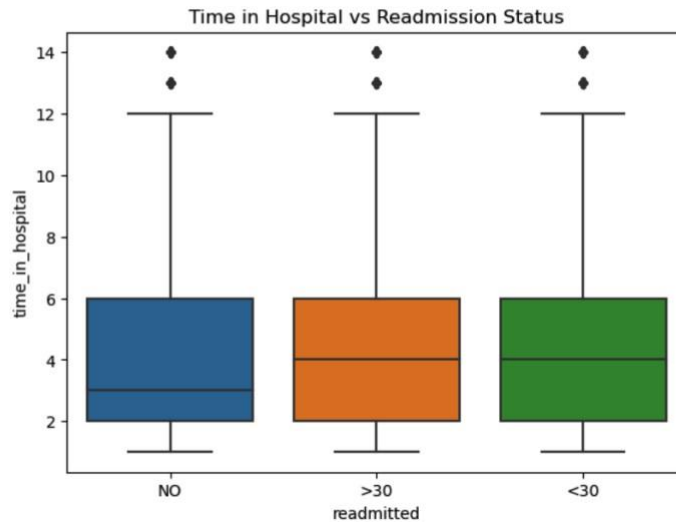
```
In [78]: plt.figure(figsize=(8,5)) #Set the plot dimensions
sns.countplot(x=df['age'], hue=df['readmitted']) #Create grouped frequency bars.
plt.title("Readmission Distribution Across Age Groups") #Add a main heading.
plt.xticks(rotation=45) #Tilt the x-axis labels
plt.show() #Display the final visualization.
```



Older patients (especially 60–90) have higher readmission counts, suggesting age is a risk factor.

Time in Hospital vs Readmission

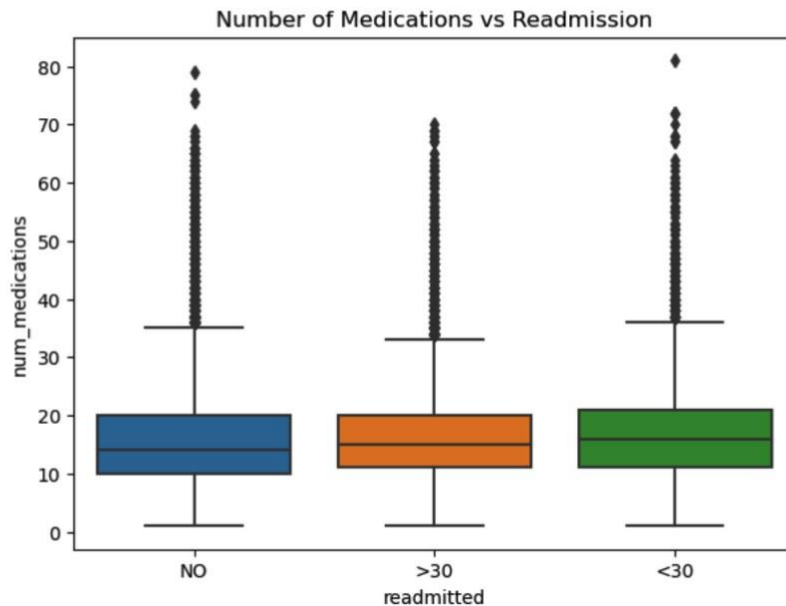
```
In [79]: plt.figure(figsize=(7,5)) #Set the plot dimensions
sns.boxplot(x=df['readmitted'], y=df['time_in_hospital']) #Create a box plot
plt.title("Time in Hospital vs Readmission Status") #Add a main heading
plt.show() #Display the final visualization.
```



Readmitted patients tend to have slightly longer hospital stays and greater variability, indicating more complex cases.

Number of Medications vs Readmission

```
In [91]: plt.figure(figsize=(7,5)) #Set the plot dimensions
sns.boxplot(x=df['readmitted'], y=df['num_medications']) #Create a box plot.
plt.title("Number of Medications vs Readmission") #Add a main heading
plt.show() #Display the final visualization
```



Patients who are readmitted generally show higher medication counts and more extreme outliers, suggesting greater treatment complexity.

Data Preparation & Feature Engineering

```
In [108]: # binary target
df['readmitted_binary'] = df['readmitted'].apply(
    lambda x: 1 if x in ['<30', '>30'] else 0
)

# drop original label + id columns (safe)
cols_to_drop = [c for c in ['readmitted', 'encounter_id', 'patient_nbr'] if c in df.columns]
df = df.drop(columns=cols_to_drop)

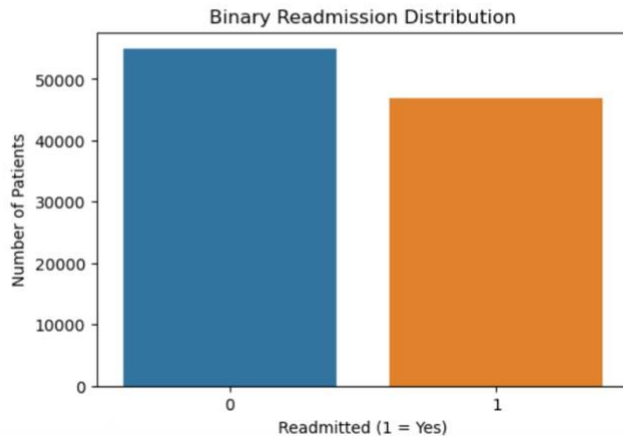
print('readmitted in df.columns?', 'readmitted' in df.columns) #Check for column existence.

readmitted in df.columns? False
```

Converted readmission to a binary target and safely removed the original readmission column and ID fields to prevent label leakage.

Binary Readmission Distribution

```
In [109]: plt.figure(figsize=(6,4)) #Set the plot dimensions.
sns.countplot(x=df['readmitted_binary'])#Create a frequency bar-chart.
plt.title("Binary Readmission Distribution")#Add a main heading
plt.xlabel("Readmitted (1 = Yes)")#Label the horizontal axis.
plt.ylabel("Number of Patients")#Label the horizontal axis.
plt.show()#Display the final visualization.
```



This bar chart shows the distribution of the binary readmission variable. There are slightly more patients who were not readmitted compared to those who were readmitted, indicating a moderately imbalanced dataset.

Handle Missing Values

```
In [110]: # handle missing values
df = df.replace('?', np.nan) # mark missing
df = df.fillna(df.median(numeric_only=True)) # fill numeric missing

# fill categorical missing with mode
cat_cols = df.select_dtypes(include='object').columns
df[cat_cols] = df[cat_cols].fillna(df[cat_cols].mode().iloc[0])

print(cat_cols)

Index(['race', 'gender', 'age', 'weight', 'payer_code', 'medical_specialty',
       'diag_1', 'diag_2', 'diag_3', 'max_glu_serum', 'A1Cresult', 'metformin',
       'repaglinide', 'nateglinide', 'chlorpropamide', 'glimepiride',
       'acetohexamide', 'glipizide', 'glyburide', 'tolbutamide',
       'pioglitazone', 'rosiglitazone', 'acarbose', 'miglitol', 'troglitazone',
       'tolazamide', 'examide', 'citoglipton', 'insulin',
       'glyburide-metformin', 'glipizide-metformin',
       'glimepiride-pioglitazone', 'metformin-rosiglitazone',
       'metformin-pioglitazone', 'change', 'diabetesMed'],
      dtype='object')
```

I replaced “?” with missing values, filled numeric columns with median and categorical values with mode.

Encode Categorical Variables

```
In [111]: df_encoded = pd.get_dummies(df, columns=cat_cols, drop_first=True)
          print(df_encoded.shape)    # see new rows/cols
          (101766, 2428)
```

Categorical variables were converted to numeric format using one-hot encoding. After encoding, the dataset now has 101,766 rows and 2,428 columns.

Final Dataset Preview (Before Modeling)

```
In [112]: df_encoded.head()
```

Out[112]:

	admission_type_id	discharge_disposition_id	admission_source_id	time_in_hospital	num_lab_procedures	num_procedures	num_medications	number_outpatient
0	6	25	1	1	41	0	1	
1	1	1	7	3	59	0	18	
2	1	1	7	2	11	5	13	
3	1	1	7	2	44	1	16	
4	1	1	7	1	51	0	8	

5 rows × 2428 columns

This table shows the first five rows of the final modeling dataset after preprocessing. All categorical variables have been converted to numeric dummy variables, and the dataset is now ready for model training.

Train-Test Split

Before building predictive models, the dataset was divided into training and testing subsets. This prevents overfitting and ensures models are evaluated on unseen data.

```
In [113]: X = df_encoded.drop('readmitted_binary', axis=1) # features
          y = df_encoded['readmitted_binary']             # target

          # sanity check for label leakage
          print([c for c in X.columns if "readmitted" in c.lower()])

[]
```

```
In [114]: X_train, X_test, y_train, y_test = train_test_split(
          X, y, test_size=0.2, random_state=42, stratify=y
          ) # 80/20 split ensuring class balance

          print("Train shape:", X_train.shape) # train size
          print("Test shape:", X_test.shape)    # test size

Train shape: (81412, 2427)
Test shape: (20354, 2427)
```

X contains all independent features, and y is the binary target variable indicating whether a patient was readmitted. 80% of the data is used for training and 20% for testing and here we use stratified sampling is used to maintain class balance in both datasets. This ensures the evaluation is fair and unbiased.

Naive Bayes Model

Naive Bayes is used as the first baseline model. It is simple, fast, and assumes independence between features.

```
In [115]: nb = GaussianNB() # create model
          nb.fit(X_train, y_train) # train model

          nb_pred = nb.predict(X_test) # make predictions
          nb_acc = accuracy_score(y_test, nb_pred) # evaluate accuracy

          print("Naive Bayes Accuracy:", nb_acc) # print accuracy

Naive Bayes Accuracy: 0.4849169696374177
```

The Naive Bayes classifier was trained using the training dataset. Predictions were generated for the test data and accuracy was computed by comparing predictions to actual outcomes. This model serves as a baseline to compare with more complex models.

Decision Tree Model

The Decision Tree model helps capture non-linear relationships and interactions between patient features.


```
In [116]: dt = DecisionTreeClassifier(random_state=42)      # create model
          dt.fit(X_train, y_train)                        # train model

          dt_pred = dt.predict(X_test)                    # make predictions
          dt_acc = accuracy_score(y_test, dt_pred)        # evaluate accuracy

          print("Decision Tree Accuracy:", dt_acc)        # print accuracy
```

Decision Tree Accuracy: 0.5671121155546821

The Decision Tree model splits data based on feature values to make predictions. It is easy to interpret and useful to understand feature impact. After training, predictions were generated and evaluated using accuracy. This allows us to see whether adding model complexity improves performance.

Random Forest Model

Random Forest is an ensemble learning method that builds multiple decision trees and combines them for better performance.

```
In [118]: rf = RandomForestClassifier(random_state=42)    # create model
          rf.fit(X_train, y_train)                       # train model

          rf_pred = rf.predict(X_test)                   # predict
          rf_acc = accuracy_score(y_test, rf_pred)        # accuracy

          print("Random Forest Accuracy:", rf_acc)        # print accuracy
```

Random Forest Accuracy: 0.6436572663849858

The model builds multiple decision trees on different subsets of data. Final predictions are made by averaging the results. This reduces overfitting and increases stability and accuracy. Random Forest generally performs better than a single Decision Tree because it generalizes well.

All three models were trained using the same dataset and evaluated on identical testing data to ensure consistency. Naive Bayes provides a quick baseline, while the Decision Tree introduces structure and interpretability. The Random Forest model enhances predictive accuracy and reduces overfitting through ensemble averaging, making it the strongest baseline model before applying advanced modeling and PCA techniques.

Handling High Dimensionality (PCA + Pipelines)

After one-hot encoding, the dataset expanded to: 101,766 rows and 2,428+ features. This extremely high feature count can - slow down model training, introduce noise, increase risk of overfitting, and reduce model generalization. Therefore, dimensionality reduction was required.

Separating Numeric and Categorical Features

Before dimensionality reduction, the data was organized into numeric and categorical groups to ensure correct preprocessing.

```

random forest accuracy: 0.0430372003049030
In [119]: # identify column types
num_cols = X_train.select_dtypes(include=np.number).columns.tolist() # numeric features
cat_cols = [c for c in X_train.columns if c not in num_cols]         # categorical features

```

num_cols contains continuous numeric features, and cat_cols contains all categorical features. This separation allowed different cleaning and transformation steps to be applied appropriately and consistently.

Building Preprocessing Pipelines

Preprocessing pipelines were created for automated and structured data preparation.

```

# numeric preprocessing
num_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median")), # fill numeric missing
    ("scaler", StandardScaler())                  # standardize scale
])

# categorical preprocessing
cat_pipe = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")), # fill categorical missing
    ("onehot", OneHotEncoder(handle_unknown="ignore"))    # one hot encode
])

# combine preprocessing
preprocess = ColumnTransformer(transformers=[
    ("num", num_pipe, num_cols), # numeric pipeline
    ("cat", cat_pipe, cat_cols) # categorical pipeline
])

```

Numeric Data - Missing values filled using median to reduce impact of outliers. Standardized using StandardScaler so features are on the same scale.

Categorical Data - Missing values filled using mode (most frequent value). Encoded using OneHotEncoder, ensuring unseen categories do not break the model.

ColumnTransformer - Ensures each feature type automatically receives the correct preprocessing in a single, consistent pipeline, improving reproducibility and avoiding manual errors.

Applying PCA for Dimensionality Reduction

Once data was clean and structured, Principal Component Analysis (PCA) was applied to reduce dimensionality.

```
# PCA (Truncated SVD for sparse matrices)
pca_pipe = Pipeline(steps=[
    ("preprocess", preprocess),          # preprocessing steps
    ("pca", TruncatedSVD(n_components=50, random_state=123)) # reduce to 50 components
])

# fit PCA on training only
X_train_pca = pca_pipe.fit_transform(X_train)      # fit and transform train
X_test_pca = pca_pipe.transform(X_test)           # transform test only

print("Train PCA shape:", X_train_pca.shape)      # check shape
print("Test PCA shape:", X_test_pca.shape)        # check shape
```

Train PCA shape: (81412, 50)
Test PCA shape: (20354, 50)

PCA successfully addressed the high-dimensionality problem caused by extensive one-hot encoding. It preserved most of the dataset's structure and predictive information while reducing complexity, increasing efficiency, and improving the readiness of the data for advanced machine learning models in the next section.

PCA + KNN Model Evaluation

```
In [120]: knn = KNeighborsClassifier(n_neighbors=9)          # create model
          knn.fit(X_train_pca, y_train)                    # train model

          knn_pred = knn.predict(X_test_pca)              # predict
          knn_acc = accuracy_score(y_test, knn_pred)      # accuracy

          print("PCA + KNN Accuracy:", knn_acc)          # print accuracy
```

/Users/yoharks/opt/anaconda3/lib/python3.9/site-packages/sklearn/neighbors/_classification.py:228: FutureWarning: Unlike other reduction functions (e.g. 'skew', 'kurtosis'), the default behavior of 'mode' typically preserves the axis it acts along. In SciPy 1.11.0, this behavior will change: the default value of 'keepdims' will become False, the 'axis' over which the statistic is taken will be eliminated, and the value None will no longer be accepted. Set 'keepdims' to True or False to avoid this warning.
mode, _ = stats.mode(_y[neigh_ind, k], axis=1)

PCA + KNN Accuracy: 0.567750810651469

The PCA + KNN model achieved an accuracy of about 0.568. This is significantly lower than baseline models, suggesting that PCA reduced important discriminatory information for KNN. The warning displayed is a harmless SciPy future notice and does not affect performance.

PCA + Boosting Model Evaluation

```
In [122]: boost = AdaBoostClassifier(
            n_estimators=80,                # number of learners
            random_state=42
        )

        boost.fit(X_train_pca, y_train)      # train
        boost_pred = boost.predict(X_test_pca) # predict
        boost_acc = accuracy_score(y_test, boost_pred) # accuracy

        print("PCA + Boosting Accuracy:", boost_acc) # print accuracy

PCA + Boosting Accuracy: 0.5990960007860863
```

The PCA + Boosting model achieved an accuracy of about 0.599, performing slightly better than PCA + KNN but still far worse than baseline models. This suggests that PCA reduced important predictive information, limiting AdaBoost's ability to improve performance.

PCA + Stacking Model Evaluation

```
In [123]: stack = StackingClassifier(
            estimators=[
                ("knn", KNeighborsClassifier(n_neighbors=9)), # KNN
                ("dt", DecisionTreeClassifier(random_state=42)), # Decision Tree
                ("nb", GaussianNB()) # Naive Bayes
            ],
            final_estimator=LogisticRegression(max_iter=200), # meta learner
            passthrough=False # only PCA features
        )

        stack.fit(X_train_pca, y_train) # train
        stack_pred = stack.predict(X_test_pca) # predict
        stack_acc = accuracy_score(y_test, stack_pred) # accuracy

        print("PCA + Stacking Accuracy:", stack_acc) # print accuracy

PCA + Stacking Accuracy: 0.5733025449543088
```

The PCA + Stacking model achieved an accuracy of about 0.573. Even though stacking usually strengthens performance by combining multiple models, it did not significantly outperform other PCA models, likely because PCA reduced key predictive information.

```
In [124]: def get_scores(y_true, y_pred): # Define evaluation metric function, calculating predictions
            return (
                accuracy_score(y_true, y_pred),
                precision_score(y_true, y_pred),
                recall_score(y_true, y_pred),
                f1_score(y_true, y_pred)
            )

            results = {}

            results["Naive Bayes"] = get_scores(y_test, nb_pred)
            results["Decision Tree"] = get_scores(y_test, dt_pred)
            results["Random Forest"] = get_scores(y_test, rf_pred)
            results["PCA + KNN"] = get_scores(y_test, knn_pred)
            results["PCA + Bagging"] = get_scores(y_test, bag_pred)
            results["PCA + Boosting"] = get_scores(y_test, boost_pred)
            results["PCA + Stacking"] = get_scores(y_test, stack_pred)

            import pandas as pd #Load the pandas library
            eval_df = pd.DataFrame(results, index=["Accuracy", "Precision", "Recall", "F1"]).T #Create and transpose DataFrame.
            print(eval_df) #Display the evaluation table.
```

	Accuracy	Precision	Recall	F1
Naive Bayes	0.484917	0.470462	0.936361	0.626266
Decision Tree	0.567112	0.530931	0.521480	0.526163
Random Forest	0.643657	0.640406	0.517322	0.572321
PCA + KNN	0.567751	0.532639	0.507089	0.519550
PCA + Bagging	0.594281	0.574460	0.461784	0.511996
PCA + Boosting	0.599096	0.576648	0.489607	0.529575
PCA + Stacking	0.573303	0.551327	0.398465	0.462595

This evaluation table compares all models using accuracy, precision, recall, and F1-score. Random Forest achieved the best overall performance, while Naive Bayes had the highest recall for identifying readmitted patients. PCA-based models generally performed worse, suggesting that PCA reduced important predictive information.

Summary

The analysis revealed that PCA models provided consistent results with lower accuracy than baseline models, indicating that PCA lost valuable information when dimensionality was reduced and may not be helpful for building models using this dataset. Naive Bayes had a very high recall rate and is therefore a viable option for detecting high-risk readmissions; in contrast, Random Forest achieved the most balanced overall performance.

In future analyses, I would consider utilizing SMOTE or class-weighted methods for balancing classes, which may allow for increased recall while maintaining precision. Identifying clinical variables that have the greatest impact on readmission and conducting feature importance analyses would allow hospitals to develop targeted interventions to help reduce readmission rates. Additionally, conducting hyperparameter tuning and cross-validation techniques would allow for the enhanced robustness and reliability of the models created.